cuGMEC Parameter Table

This document corresponds to `src/cuGMEC_param.h`. You must recompile after changing these parameters.

The Example column shows only one possible way to write each parameter; it does not imply a recommended value.

Notation:

Notation	Meaning
`<Species>`	`Ion`, `Alpha`, `Beam`
`<MHDField>`	`Phi`, `A`, `dNe`, `dTe`
`<PICField>`	`dPi`, `dPa`, `dPb`

Data Types and File Paths

Parameter	Allowed Values	Description	Notes	Example
`mhdReal`	`float` / `double`	MHD calculation precision.	MHD precision should be greater than or equal to PIC precision.	`using mhdReal = double;`
`picReal`	`float` / `double`	PIC calculation precision.	PIC precision should be less than or equal to MHD precision.	`using picReal = float;`
`inputDir`	String path	Input file directory.	For a run from scratch, only the `MHDEquilibrium` file is needed. For a continued run, the `MHDContinue` and `PICContinue` files are also needed. They come from the previous run's `outputDir/final`.	`const std::string inputDir = "/path/to/input";`
`outputDir`	String path	Output file directory.		`const std::string outputDir = "/path/to/output";`
`MHDEquilibrium`	`.bin` file name	MHD equilibrium file.	Generated by `generateInput2D.m` for tokamaks and `generateInput3D.m` for stellarators.	`const std::string MHDEquilibrium = "MHDEquilibrium_384_32.bin";`
`<Species>PhaseSpaceMapping`	`.bin` file name	Phase-space mapping file.	Required only when `ifOutputPhaseSpace*` is enabled; generated by `generatePhaseSpaceMapping2D.m`.	`const std::string IonPhaseSpaceMapping = "IonPhaseSpaceMapping.bin";`

Normalization

Parameter	Allowed Values	Description	Notes	Example
`B0`	Real number	Magnetic-field normalization scale.		`const double B0 = 4.921751144619735;`
`L0`	Real number	Length normalization scale.		`const double L0 = 6.595629295925759;`
`VA0`	Real number	Velocity normalization scale.		`const double VA0 = 8.864164667700194e+06;`
`RHO0`	Real number	Left endpoint of the radial interval.		`const double RHO0 = 0.08;`
`RHO1`	Real number	Right endpoint of the radial interval.		`const double RHO1 = 0.90;`
`PSITMAX`	Real number	Outermost toroidal magnetic flux divided by 2π (Wb/rad).		`const double PSITMAX = 18.868213504765762;`

Grid and Parallel Setup

Parameter	Allowed Values	Description	Notes	Example
`hostNums`	Integer	Number of MPI processes.		`const int hostNums = 4;`
`devNums`	Integer	Number of GPUs used by each MPI process.		`const int devNums = 4;`
`gridNx`	Integer	Number of radial grid points.		`const int gridNx = 256;`
`gridNy`	Integer	Number of field-aligned grid points.	Must be divisible by `hostNums * devNums`.	`const int gridNy = 32;`
`gridNz`	Integer	Number of toroidal grid points.		`const int gridNz = 96;`
`NFP`	Integer	Number of field periods. `NFP=1` for tokamak; `NFP!=1` for stellarator.		`const int NFP = 1;`
`ppcNums`	Integer	Average number of particles per species in each grid cell.		`const int ppcNums = 256;`

Toroidal Range and Initial Perturbation

Parameter	Allowed Values	Description	Notes	Example
`tubes`	Integer	Simulate `1/tubes` of the toroidal domain.		`const int tubes = 6;`
`leftN`	Integer	Lower bound of the retained toroidal mode number in the `1/tubes` toroidal domain.		`const int leftN = 1;`
`rightN`	Integer	Upper bound of the retained toroidal mode number in the `1/tubes` toroidal domain.	Retained toroidal modes are `tubes*[leftN,rightN]`, spaced by `tubes`.	`const int rightN = 6;`
`refinedTimes`	Integer	Poloidal-grid refinement factor for `selectNM_*` filtering.	The refined poloidal grid size is `gridNy * refinedTimes`.	`const int refinedTimes = 32;`
`perturbLeftN`	Integer	Lower bound of the initial perturbation toroidal mode number in the `1/tubes` toroidal domain.		`const int perturbLeftN = 1;`
`perturbRightN`	Integer	Upper bound of the initial perturbation toroidal mode number in the `1/tubes` toroidal domain.	Initial perturbation modes are `tubes*[perturbLeftN,perturbRightN]`, spaced by `tubes`.	`const int perturbRightN = 6;`
`perturbRadialIndex`	Integer	Radial peak grid point of the initial Gaussian perturbation.		`const int perturbRadialIndex = 85;`
`perturbWidth`	Real number	Radial width of the initial Gaussian perturbation.		`const mhdReal perturbWidth = 0.04;`
`perturbAmplitude`	Real number	Normalized `Phi` amplitude of the initial Gaussian perturbation.	The perturbation is proportional to `amplitude * exp[-(X - Xc)^2 / (2 * width^2)]`, where `Xc = (radialIndex - 1) / (gridNx - 1)`.	`const mhdReal perturbAmplitude = 2.5e-9;`

Filtering

Parameter	Allowed Values	Description	Notes	Example
`ifFilterN_<MHDField>`	`trueType` / `falseType`	Whether to apply toroidal filtering to the MHD field.		`using ifFilterN_Phi = trueType;`
`ifFilterN_dP`	`trueType` / `falseType`	Whether to apply toroidal filtering to the PIC perturbed pressure.	Applies to `dPi/dPa/dPb`.	`using ifFilterN_dP = trueType;`
`removeN_<MHDField>`	`{}` or list of toroidal mode numbers	Remove the specified toroidal mode numbers from the MHD field.		`constexpr std::array<int, 1> removeN_Phi = {7};`
`removeN_dP`	`{}` or list of toroidal mode numbers	Remove the specified toroidal mode numbers from the PIC perturbed pressure.	Applies to `dPi/dPa/dPb`.	`constexpr std::array<int, 0> removeN_dP = {};`
`selectNM_<MHDField>`	`{{{N, leftM, rightM}, ...}}`	Apply poloidal filtering to the specified toroidal mode numbers of the MHD field.		`constexpr std::array<std::tuple<int, int, int>, 1> selectNM_Phi = {{{0, 0, 0}}};`
`selectNM_dP`	`{{{N, leftM, rightM}, ...}}`	Apply poloidal filtering to the specified toroidal mode numbers of the PIC perturbed pressure.	Applies to `dPi/dPa/dPb`.	`constexpr std::array<std::tuple<int, int, int>, 1> selectNM_dP = {{{0, 0, 1}}};`

MHD Physics Parameters

Parameter	Allowed Values	Description	Notes	Example
`ifNonlinearMHD`	`trueType` / `falseType`	Whether to include MHD nonlinear terms.		`using ifNonlinearMHD = trueType;`
`ifDiaDrift`	`trueType` / `falseType`	Whether to include ion diamagnetic drift in the vorticity equation.		`using ifDiaDrift = trueType;`
`ifEparallel`	`trueType` / `falseType`	Whether to include the parallel electric-field term.		`using ifEparallel = trueType;`
`ifFLRMHD`	`trueType` / `falseType`	Whether to include the FLR effect from background thermal ions in the Poisson equation.		`using ifFLRMHD = falseType;`
`ifQNeutrality`	`trueType` / `falseType`	Whether to compute the electron density perturbation from quasi-neutrality.	When enabled, `dNe` is determined by vorticity and the ion density perturbation.	`using ifQNeutrality = trueType;`
`ifReducedMHD`	`trueType` / `falseType`	Evolves the pressure of the background thermal plasma with a single-fluid pressure equation; disables evolution of `dNe` and `dTe`.	Use with `ifEparallel=falseType`.	`using ifReducedMHD = falseType;`
`Gamma`	Real number	Adiabatic index in the single-fluid pressure equation.		`const mhdReal Gamma = 0.0;`
`ifMaxwellStress`	`trueType` / `falseType`	Whether to include Maxwell stress.	Effective only when `ifNonlinearMHD=trueType`.	`using ifMaxwellStress = trueType;`
`ifReynoldsStress`	`trueType` / `falseType`	Whether to include Reynolds stress.	Effective only when `ifNonlinearMHD=trueType`.	`using ifReynoldsStress = trueType;`
`MaxwellStressCoef`	Real number	Maxwell stress coefficient.	Effective only when `ifMaxwellStress=trueType`.	`const mhdReal MaxwellStressCoef = 1.0;`
`ReynoldsStressCoef`	Real number	Reynolds stress coefficient.	Effective only when `ifReynoldsStress=trueType`.	`const mhdReal ReynoldsStressCoef = 1.0;`

Dissipation

Parameter	Allowed Values	Description	Notes	Example
`ifNablaPerp2<MHDField>`	`trueType` / `falseType`	Enable second-order perpendicular MHD-field dissipation.		`using ifNablaPerp2Phi = trueType;`
`perp2<MHDField>`	Real number	Coefficient for second-order perpendicular MHD-field dissipation.		`const mhdReal perp2Phi = 1.0e-7;`
`ifNablaPara4<MHDField>`	`trueType` / `falseType`	Enable fourth-order parallel MHD-field dissipation.		`using ifNablaPara4Phi = trueType;`
`para4<MHDField>`	Real number	Coefficient for fourth-order parallel MHD-field dissipation.		`const mhdReal para4Phi = 1.0e-6;`
`ifNablaPerp2dP`	`trueType` / `falseType`	Enable second-order perpendicular PIC-pressure dissipation.	Applies to `dPi/dPa/dPb`.	`using ifNablaPerp2dP = trueType;`
`perp2dP`	Real number	Coefficient for second-order perpendicular PIC-pressure dissipation.		`const mhdReal perp2dP = 1.0e-6;`
`ifNablaPara4dP`	`trueType` / `falseType`	Enable fourth-order parallel PIC-pressure dissipation.	Applies to `dPi/dPa/dPb`.	`using ifNablaPara4dP = falseType;`
`para4dP`	Real number	Coefficient for fourth-order parallel PIC-pressure dissipation.		`const mhdReal para4dP = 0.0;`

PIC Physics Parameters

Parameter	Allowed Values	Description	Notes	Example
ifFLRPIC	trueType / falseType	Whether to enable the FLR effect in the PIC component.		using ifFLRPIC = trueType;
ifNonlinearPIC	trueType / falseType	Whether to enable PIC nonlinear terms.		using ifNonlinearPIC = trueType;
gyroNums	Integer	Number of gyro-average points.	Must satisfy gyroNums = 2^n with n >= 2.	const int gyroNums = 4;

PIC Species Parameters

Parameter	Allowed Values	Description	Notes	Example
if<Species>	trueType / falseType	Whether to enable the corresponding ion species.		using ifIon = trueType;
<Species>Type	Maxwell / Slowing0 / Slowing1 / Slowing2 / Slowing3	Velocity distribution type.		const disType IonType = Maxwell;
<Species>Space	spaceReal / spaceUniform	Marker-position sampling: physical spatial distribution or uniform.		const spaceType IonSpace = spaceReal;
<Species>Velocity	velocityReal / velocityUniform	Marker-velocity sampling: physical velocity distribution or uniform.		const velocityType IonVelocity = velocityUniform;
<Species>Mass	Real number	Ion mass, normalized by the proton mass.		const picReal IonMass = 2.5;
<Species>Char	Real number	Ion charge, normalized by the electron charge.		const picReal IonChar = 1.0;
<Species>Beta	Real number	On-axis ion beta (P/(B0^2/(2*mu0))).		const picReal IonBeta = 0.037793721898356;
<Species>Vmin	Real number	Lower bound of the ion velocity, normalized by VA0.		const picReal IonVmin = 0.0135;
<Species>Vmax	Real number	Upper bound of the ion velocity, normalized by VA0.		const picReal IonVmax = 0.54;
<Species>Vb	Real number	Cutoff velocity of the slowing-down distribution, normalized by VA0.		const picReal BeamVb = 0.1;
<Species>DeltaV	Real number	Cutoff width of the slowing-down distribution, normalized by VA0.		const picReal BeamDeltaV = 0.1;
<Species>Lambda0	Real number	Center of Lambda in the anisotropic distribution. Lambda = mu*B0/E.		const picReal BeamLambda0 = 0.4;
<Species>DeltaLambda2	Real number	Square of the Lambda width in the anisotropic distribution. Lambda = mu*B0/E.		const picReal BeamDeltaLambda2 = 1.0 / (4.5 * 4.5);

Distribution Type	Formula
Maxwell	f ~ n * T^(-3/2) * exp[-m*V^2/(2*T)]
Slowing0	f ~ n / (V^3 + Vc^3)
Slowing1	f ~ n * [1 + erf((Vb - V)/DeltaV)] / (V^3 + Vc^3)
Slowing2	f ~ n * exp[-(Lambda - Lambda0)^2 / DeltaLambda^2] / (V^3 + Vc^3)
Slowing3	f ~ n * [1 + erf((Vb - V)/DeltaV)] * exp[-(Lambda - Lambda0)^2 / DeltaLambda^2] / (V^3 + Vc^3)

Diagnostics

Parameter	Allowed Values	Description	Notes	Example
ifDiagAmplitude	trueType / falseType	Whether to diagnose mode amplitude.		using ifDiagAmplitude = trueType;
ifDiagFrequency	trueType / falseType	Whether to diagnose mode frequency.		using ifDiagFrequency = trueType;
ifDiagEparallel	trueType / falseType	Whether to diagnose the parallel electric field.		using ifDiagEparallel = trueType;
ifDiagDensity	trueType / falseType	Whether to diagnose the ion density perturbation.		using ifDiagDensity = trueType;
ifDiagDiffusivity	trueType / falseType	Whether to diagnose ion diffusivity.		using ifDiagDiffusivity = trueType;
ifDiagZFDrive	trueType / falseType	Whether to diagnose the zonal-flow drive source.		using ifDiagZFDrive = falseType;
ifCheckNAN	trueType / falseType	Whether to check for NaN.	If NaN is diagnosed, the program stops immediately and writes output.	using ifCheckNAN = trueType;

MHD Field Output

Parameter	Allowed Values	Description	Notes	Example
`ifOutputPhi`	`trueType` / `falseType`	Whether to output `Phi` at multiple time points.		`using ifOutputPhi = trueType;`
`ifOutputA`	`trueType` / `falseType`	Whether to output `A` at multiple time points.		`using ifOutputA = falseType;`
`ifOutputdNe`	`trueType` / `falseType`	Whether to output `dNe` at multiple time points.		`using ifOutputdNe = falseType;`
`ifOutputdTe`	`trueType` / `falseType`	Whether to output `dTe` at multiple time points.		`using ifOutputdTe = falseType;`
`ifOutputdP`	`trueType` / `falseType`	Whether to output the single-fluid pressure `dP` at multiple time points.		`using ifOutputdP = falseType;`
`ifOutputdPi`	`trueType` / `falseType`	Whether to output `dPi` at multiple time points.		`using ifOutputdPi = falseType;`
`ifOutputdPa`	`trueType` / `falseType`	Whether to output `dPa` at multiple time points.		`using ifOutputdPa = falseType;`
`ifOutputdPb`	`trueType` / `falseType`	Whether to output `dPb` at multiple time points.		`using ifOutputdPb = falseType;`

Phase-space Output

Parameter	Allowed Values	Description	Notes	Example
`gridE`	Integer	Number of phase-space grid points in the `E` direction.	Must match the settings in `generatePhaseSpaceMapping2D.m`.	`const int gridE = 96;`
`gridPphi`	Integer	Number of phase-space grid points in the `Pphi` direction.	Must match the settings in `generatePhaseSpaceMapping2D.m`.	`const int gridPphi = 128;`
`gridLambda`	Integer	Number of phase-space grid points in the `Lambda` direction.	Must match the settings in `generatePhaseSpaceMapping2D.m`.	`const int gridLambda = 48;`
`ppcPhase`	Integer	Average number of particles per species in each phase-space grid cell.		`const int ppcPhase = 2048;`
`ifOutputPhaseSpaceJacobian`	`trueType` / `falseType`	Whether to output the phase-space Jacobian.		`using ifOutputPhaseSpaceJacobian = trueType;`
`ifOutputPhaseSpaceOrbit`	`trueType` / `falseType`	Whether to output phase-space orbit frequencies.		`using ifOutputPhaseSpaceOrbit = falseType;`
`ifOutputPhaseSpaceF0`	`trueType` / `falseType`	Whether to output the phase-space equilibrium distribution function.		`using ifOutputPhaseSpaceF0 = falseType;`
`ifOutputPhaseSpaceDeltaF`	`trueType` / `falseType`	Whether to output the phase-space perturbed distribution function.		`using ifOutputPhaseSpaceDeltaF = falseType;`
`ifOutputPhaseSpacePower`	`trueType` / `falseType`	Whether to output phase-space wave-particle interaction power.		`using ifOutputPhaseSpacePower = falseType;`
`gridVpara`	Integer	Number of pitch-space grid points in the `vpara` direction.		`const int gridVpara = 128;`
`gridVperp`	Integer	Number of pitch-space grid points in the `vperp` direction.		`const int gridVperp = 64;`
`ppcPitch`	Integer	Average number of particles per species in each pitch-space grid cell.		`const int ppcPitch = 2048;`
`ifOutputPitchSpaceJacobian`	`trueType` / `falseType`	Whether to output the pitch-space Jacobian.		`using ifOutputPitchSpaceJacobian = trueType;`
`ifOutputPitchSpaceF0`	`trueType` / `falseType`	Whether to output the pitch-space equilibrium distribution function.		`using ifOutputPitchSpaceF0 = falseType;`
`ifOutputPitchSpaceDeltaF`	`trueType` / `falseType`	Whether to output the pitch-space perturbed distribution function.		`using ifOutputPitchSpaceDeltaF = falseType;`
`ifOutputPitchSpacePower`	`trueType` / `falseType`	Whether to output pitch-space wave-particle interaction power.		`using ifOutputPitchSpacePower = falseType;`

Time Advancement

Parameter	Allowed Values	Description	Notes	Example
`ifContinue`	`trueType` / `falseType`	Whether to start from continue files.	The `inputDir` directory must contain the `MHDContinue` and `PICContinue` continue files. They come from the previous run's `outputDir/final`.	`using ifContinue = falseType;`
`continueSteps`	Integer	Number of steps already completed before resuming.	Must match the suffix of the continue file names.	`const int continueSteps = 0;`
`dt`	Real number	MHD time step.	Normalized by the Alfven time `L0/VA0`.	`const double dt = 0.02;`
`totalSteps`	Integer	Total number of MHD steps in this run.		`const int totalSteps = 20000;`
`ratioDt`	Integer	Ratio of the PIC time step to the MHD time step.	PIC uses `dt * ratioDt` for each push.	`const int ratioDt = 1;`
`sortSteps`	Integer	Particle sorting interval.		`const int sortSteps = 25;`
`diagSteps`	Integer	Diagnostic sampling interval.		`const int diagSteps = 1;`
`outputSteps`	Integer	Field and phase-space output interval.		`const int outputSteps = 2500;`